Example repository

README MIT license

Bulletproof frontend testing

This repository supports a series of talks and workshops focused on testing frontend applications the bulletproof way. Its aim is to provide testing suites, cases and patterns as well as rich explanations of their rationales.

It contains an example React SPA with a thin layer of supporting backend. Testing suites cover integration, functionality, accessibility, visual regression and so on.

The talks were performed during:

- Voxxed Days Crete conference talk (26-27.09.2025 - Heraklion, Crete Island, Greece)
- React Alicante workshop (02-04.10.2025 - Alicante, Spain)

https://github.com/LukaszNowakPL/bulletproof-frontend-testing

Bulletproof frontend testing

Łukasz Nowak, React Alicante 2026

Alicante, 24 September 2026

Let’s introduce ourselves



Łukasz Nowak

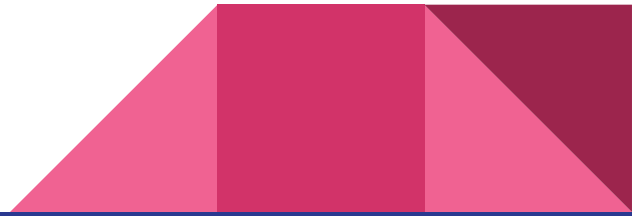
Señor frontend dev

Łukasz Nowak

LukaszNowakPL

Agenda-like for workshops

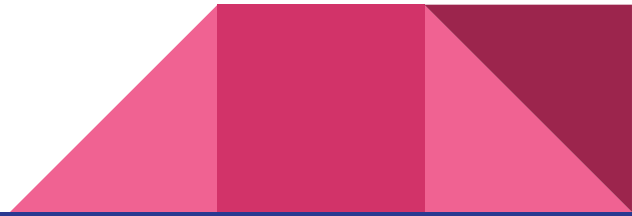
- Integration testing the bulletproof way
  - Mistakes, pitfalls, misunderstandings and all bad things
  - Necessities to make it right way
  - Running through view test cases
  - Investigating additional test cases and enterprise class tips
  - Migration to Vitest browser mode
- PhD in api mocking
  - What to avoid to make it good
  - Factory pattern and randomization to make it pro+
- Playwright testing
  - Functional / e2e testing
  - Accessibility testing
  - Snapshot testing



Integration testing

Why do we suck hard on tests?

- They sometimes bring no value
- We don’t understand proper tests approaches …
- … and literature sometimes doesn’t help much
- They are tightly coupled with the source code
- Sooner or later, they become flaky
- Because we tend to overcomplicate them



What integration tests mean?

- Development related ones:
  - Direct checks of non-compiled code
  - Simple, component-related scenarios and quick answers
- Based on Vitest ecosystem & Testing Library family (so far)
- Playwright and Cypress are cool, but not for this part of a workshop



Airports dashboard

Airports Add airport

Dark mode

Add airport

Name

Test airport name

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

”

You need only four test cases
to check such a page completely.

Łukasz Nowak, React Alicante 2026

Mistakes we usually do

Airports dashboard

Airports Add airport

Dark mode

Add airport

Name

Airport name is required

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

Test with false-positive outcome

```
it('does not display validation error by default' , async () => {
   // When component is rendered
   render(<AddAirportView />)

   // Then the validation message is not displayed
   expect(screen.queryByText('Airport name is required' ))
      .not.toBeInTheDocument ();
});
```



Airports dashboard

Airports Add airport

Dark mode

Fixed test case

```
it('does not display validation error by default' , async () => {
    // When component is rendered
    render(<AddAirportView />)

    // And data loader appeared only during data fetching
    expect(screen.getByTestId('progress-bar')).toBeInTheDocument();
    await waitForElementToBeRemoved(screen.queryByTestId('progress-bar'))

    // Then the validation message is not displayed
    expect(screen.queryByText('Airport name is required'))
        .not.toBeInTheDocument();
});
```

Airports dashboard

Airports Add airport

Dark mode

Sorry, there is some connectivity error

We were unable to fetch some of data necessary to display the page. You can restart data fetching manually.
If it won't help, try to restart the app.

Airports dashboard

Airports Add airport

Dark mode

Add airport

Name

Airport name is required

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

Testing implementation details instead of functionality

```
// src/const/routes.ts
const ROUTES = {
   AIRPORTS: '/airports'
} as const
```

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
   const {history} = render(<AddAirportView />)

   // Form fulfillment...

   await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
   expect(history.location.pathname).toEqual(ROUTES.AIRPORTS)
});
```

How such a test can fire back?

```
// src/const/routes.ts
const ROUTES = {
   AIRPORTS: '/assports'
} as const
```

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
   const {history} = render(<AddAirportView />)

   // Form fulfillment...

   await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
   expect(history.location.pathname).toEqual(ROUTES.AIRPORTS)
});
```

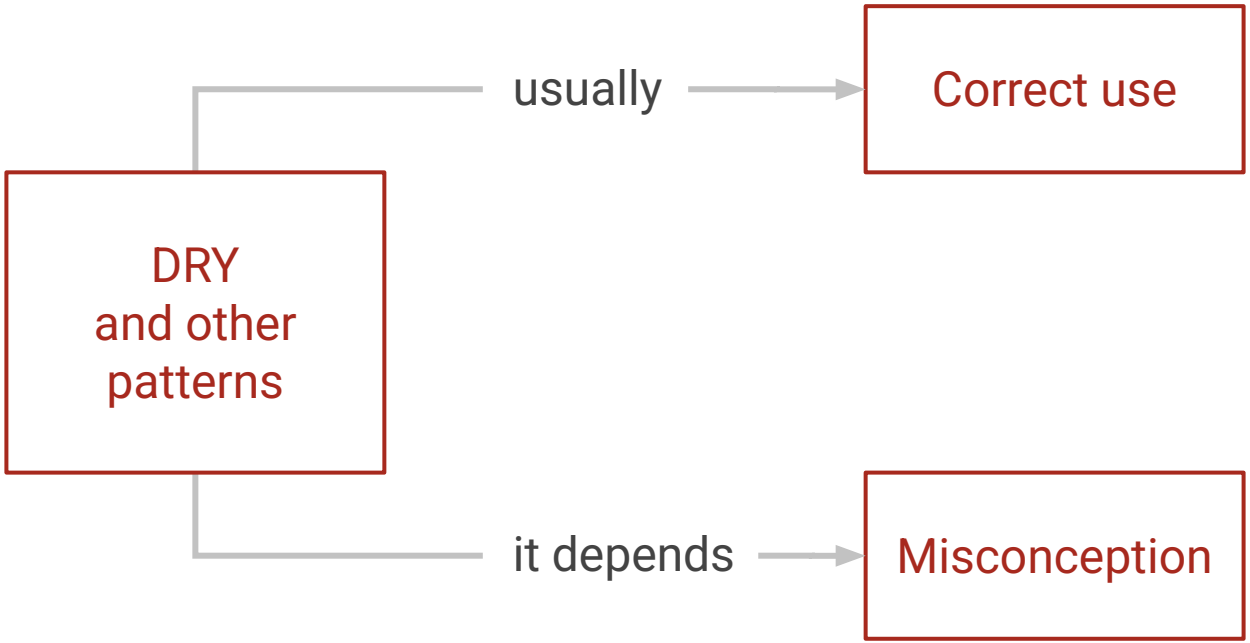
Testing functionality instead of implementation details

```
// tests/AddAirportView.test.tsx
it('redirects to /airports after new airport creation' , async () => {
   const {history} = render(<AddAirportView />)

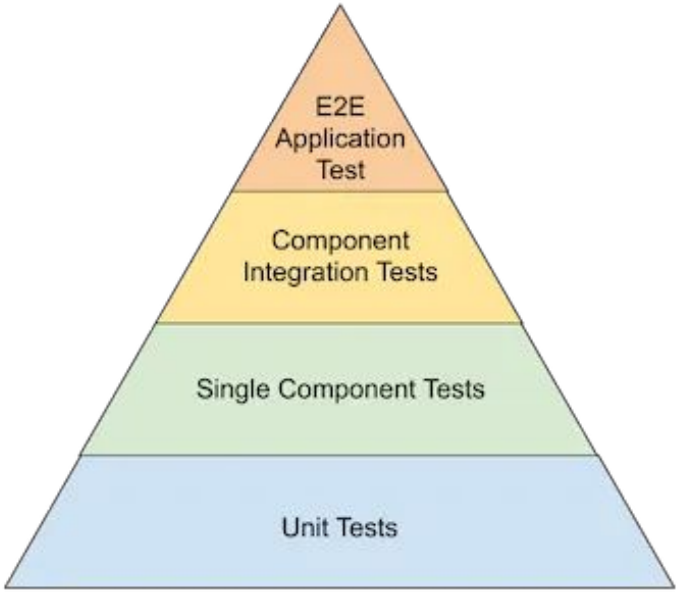
   // Form fulfillment...

   await userEvent.click(screen.getByRole('button', {name: 'Submit'}))
   expect(history.location.pathname).toEqual('/airports')
});
```

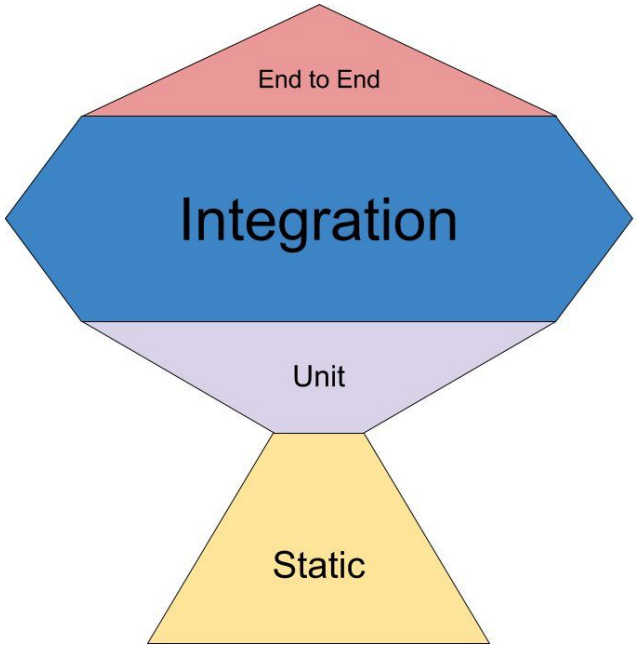
Where can we get to using good patterns?



Testing levels misconception



Source: https://medium.com/inspiredbrilliance/testing-single-page-applications-b4001adc6452



Source: https://x.com/kentcdodds/status/960723172591992832

”

Frontend app tests should align with a frontend ecosystem.

What may happen if we follow books literally?

Source: https://www.youtube.com/watch?v=zE7PKRjrid4

Airports dashboard

Airports Add airport

Dark mode

Add airport

Name

Airport name is required

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

#1 Testing too low with redundant mocking

```
// tests/AddAirportView/Name.test.tsx
it('displays message when validation return error' , async () => {
    // Given form hook mocked with error being on
    vi.mocked(useField).mockReturnValue({
        meta: {dirty: true, error: 'Test error'}
    } as FieldRenderProps<string>)

    // When component is rendered with error state
    render(<Name />)

    // Then error message is displayed
    expect(screen.getByText('Test error')).toBeInTheDocument()
})
```

#2 Lack of idea of what should get tested

```
export const AddAirportView = () => {
    return (<>
        <FormContainer>
            <Name />
            <IataCode />
            <Country />
            <Regions />
            <VaccinationNotes />
        </FormContainer>
    </>)
}
```

#3 Application enhancement example

```
export const AddAirportView = () => {
    return (<>
        <Collapsible header={'Important!'}>
            <p>Important note content</p>
        </Collapsible>
        <FormContainer>
            <Name />
            <IataCode />
            <Country />
            <Regions />
            <VaccinationNotes />
        </FormContainer>
    </>)
}
```

```
// tests/AddAirportView.test.tsx
it("prints note header and displays content only after click" , async () => {
    // When component render
    render(<AddAirportView />)

    // Then only message header is displayed
    expect(await screen.findByRole('heading', {name: "Important!"}))
        .toBeInTheDocument()
    expect(screen.queryByText('Important note content'))
        .not.toBeInTheDocument()

    // When user click on collapsible element
    await userEvent.click(screen.getByRole('heading', {name: "Important!"}))

    // Then full content is displayed
    expect(await screen.findByText('Important note content'))
        .toBeInTheDocument()
})
```

Refactor opportunity

```
export const AddAirportView = () => {
    return (<>
        <ImportantMessage />
        <FormContainer>
            <Name />
            <IataCode />
            <Country />
            <Regions />
            <VaccinationNotes />
        </FormContainer>
    </>)
}
```

#4 Which causes mess

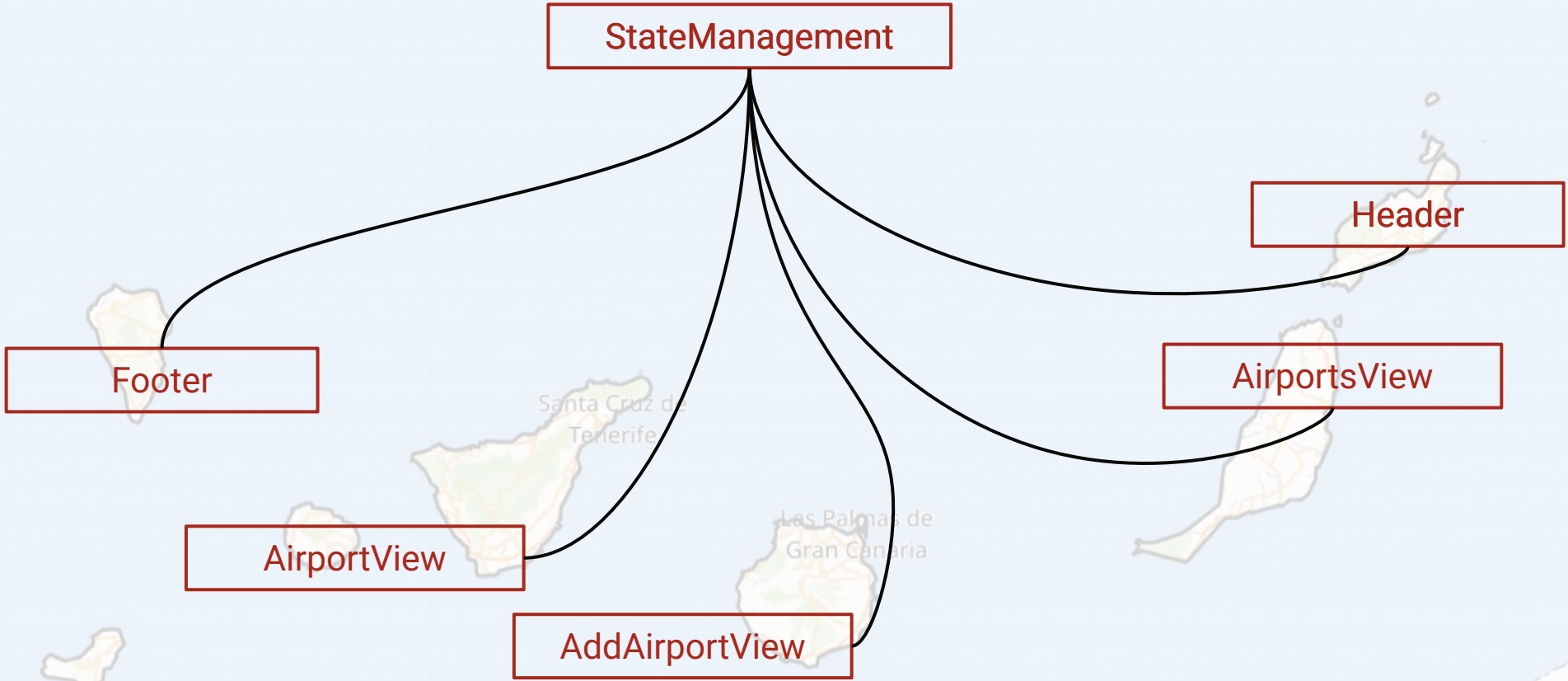
```
export const AddAirportView = () => {
    return (<>
        <ImportantMessage />
        {/* Component has been moved, yet tests remain in the same file */}
        <FormContainer>
            <Name />
            <IataCode />
            <Country />
            <Regions />
            <VaccinationNotes />
        </FormContainer>
    </>)
}
```

Doing it the right way

Doing it the right way

Island architecture pattern




StateManagement
Header
Footer
AirportsView
AirportView
AddAirportView

Island architecture pattern with React Router

```
export const routerDefinition = [
    {
        path: '/', element: <App />, // Lanzarote & La Palma
        children: [
            {
                path: '/airports',
                children: [
                    { index: true, element: <AirportsView />}, // Fuerteventura
                    { path: 'add', element: <AddAirportView />}, // Gran
Canaria
                    { path: ':id', element: <AirportView />}, // Tenerife
                ],
            },
        ],
    },
] satisfies RouteObject[];
```

First attempt to integration tests

```
Error: No QueryClient set, use QueryClientProvider to set one
 ❯ useQueryClient @tanstack/react-query/build/modern/QueryClientProvider.js
 ❯ useBaseQuery @tanstack/react-query/build/modern/useBaseQuery.js
 ❯ useQuery @tanstack/react-query/build/modern/useQuery.js
 ❯ useAirports src/hooks/useAirports.ts
      5| export const useAirports = () => {
      6|   return useQuery({queryFn: getAirports, …});
      7| };
       | ^
 ❯ AddAirportViewGuard src/views/AddAirportView/AddAirportView.guard.tsx
 ❯ renderWithHooks react-dom/cjs/react-dom.development.js
...
```

We won’t quit dependencies

```
export const App: React.FC = () =>
   <DesignSystemTheme>
       <AppState>
           <ReactQueryContext>
               <AppLayout>
                   <Outlet />
               { /* ^ either AirportsView, AirportView or AddAirportView */}
               </AppLayout>
           </ReactQueryContext>
       </AppState>
   </DesignSystemTheme>
```

Doing it the right way

Component wrapper pattern

Component wrapper implementation

```
export const renderWithContexts = (component, options = {}) => {
    // To include React Router
    const {path = '/', url = ''} = options;
    const history = createMemoryHistory({initialEntries: [url]});

    // To include TanStack Query
    const queryClient = new QueryClient();

    return render(<ReactQueryContext queryClient={queryClient}>
        <HistoryRouter history={history}>
            <Routes>
                <Route path={path} element={component} />
            </Routes>
        </HistoryRouter>
    </ReactQueryContext>)
};
```

Test of a wrapped component

```
✓ test/integration/Views/AddAirportView.test.tsx (1 test)  1405ms
  ✓ AddAirportView (1)
    ✓ Happy path (1)
      ✓ renders component and adds airport  1402ms

Test Files  1 passed (1)
     Tests  1 passed (1)
  Start at  10:10:37
  Duration  5.17s
```

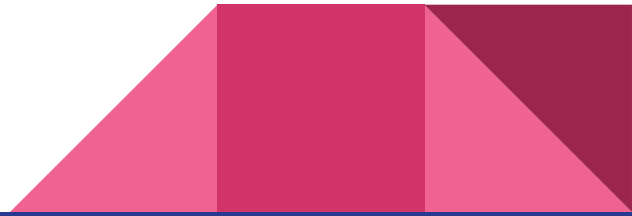
How about performance impact?

```
 RUN  v3.2.4
      Running tests with seed "1757753204226"
      Coverage enabled with istanbul

 ✓ test/integration/test-scenarios/AppFooter.test.tsx  (2 tests) 31ms
   ✓ AppFooter (2)
     ✓ renders raw AppFooter component  11ms
     ✓ renders AppFooter component wrapped in component wrappers  20ms
```

Rendering entire App means

- Handling api calls not related with tested view
- Additional navigation to a tested view
- Displaying data being part of other route views
  (nasty edge case of nested route views like ‘/countries/12/airports/21/airlines’)
- Complications with stepper user journeys
  (go through first n-1 steps to test nth step)



Doing it the right way

Integration tests

What is integration tests suite all about?



Kent C. Dodds ⚡
@kentcdodds

The more your tests resemble the way your software is used, the more confidence they can give you.

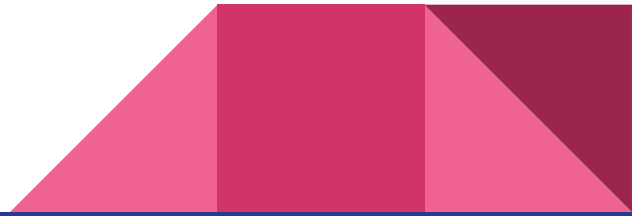
Przetłumacz wpis

4:05 AM · 23 mar 2018

17 340 1 tys. 65

What frontends integrate with?

- Backend services - api calls
  (handled by TanStack Query, mimicked via MSW)
- View display logic - via url (handled by React Router)
- User - data display or interactions via buttons, links, form controls
  (handled by HTML/JS, mimicked via Testing Library family)



Doing it the right way

Effective test scenarios

Add airport

Name

Test airport name

IATA Code

TES

Country

Greece

Regions

island

- [ ] Aegean Islands
- [ ] Balearic Islands
- [ ] Canary Islands
- [ ] Corfu Island (Kerkyra)
- [x] Crete Island
- [ ] Dodecanese Islands
- [ ] Ha Long Island
- [ ] Ionian Islands
- [ ] Maiorca Island
- [ ] Rhodes Island
- [ ] Tenerife Island

Vaccination Notes

Submit

Happy and negative paths approach

```
describe('AddAirportsView', () => {
    describe('Happy path', () => {
        // Any scenario leading to functional success:
        // - all api calls responding 200
        // - shape of data sent / received is ok
        // - everything is displayed and work as expected
    }),
    describe('Negative path', () => {
        // Anything stopping us from adding new airport:
        // - some endpoint responded with error
        // - user provided invalid data
    })
});
```

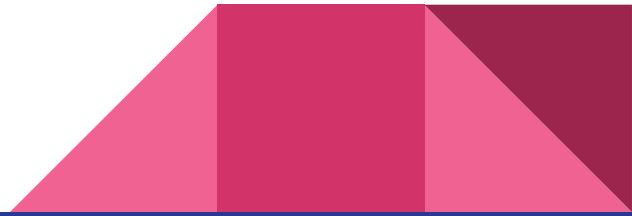
Test cases for AddAirportView

```
describe('AddAirportsView', () => {
    describe('Happy path', () => {
        it('renders view correctly and adds airport' , () => {})
    })
    describe('Negative path', () => {
        it('displays fetch error component on GET api fail' , () => {})
        it('renders validation error' , () => {})
        it('displays error message on POST api fail' , () => {})
    })
})
```

Investigation of integration test cases

Test cases run through

- Integration tests
  - Grouping, use of Gherkin notation, api mocking
  - renderWithContexts() (a component wrapper) explanation
  - Test scenarios run through
- State management testing
  - Purpose of dedicated dashboard
  - Rationale behind monolith pattern
- Routing configuration testing
  - View mocking approach
  - Test cases run through



Pro+ tips & tricks

- data-test-id being bad pattern
- No console logs assertion
- File parallelism
- Sequence shuffling



Vitest browser mode

- Better than JSDom as it uses real browser and actions
- Provides own utilities - no need to use RTL anymore
- Provides dashboard for better tests debugging
- Is not production ready yet
- Migration is not that easy peasy
- Done on the branch of example repo



Api mocking practices

Dedicated repository
(for api mocking)

README MIT license

Advanced api mocking patterns

This repository supports `Advanced patterns of api mocking in tests` talk. Its aim is to explain several most common mistakes on tests of applications/components triggering api endpoint calls. It also contains example of factory pattern and data randomization to better reflect unpredictable way users use the application.

This talk was performed during:

- Online track of Warszawskie Dni Informatyki conference (04.04.2025)

Project structure

The project contains React frontend application and supporting backend stub service based on `json-server` component.

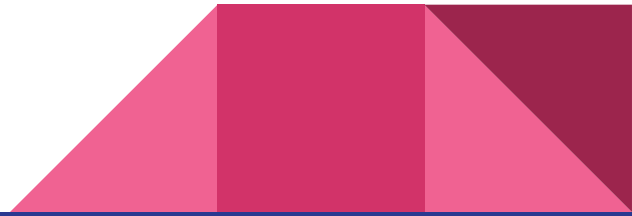
Test examples supporting each chapter of the talk are groupped on /frontend/test directory. To cover both - developer and Tester/QA/SDET - approach, each chapter contains Playwright functional examples as well as Vitest integration examples.

https://github.com/LukaszNowakPL/advanced-api-mocking-patterns

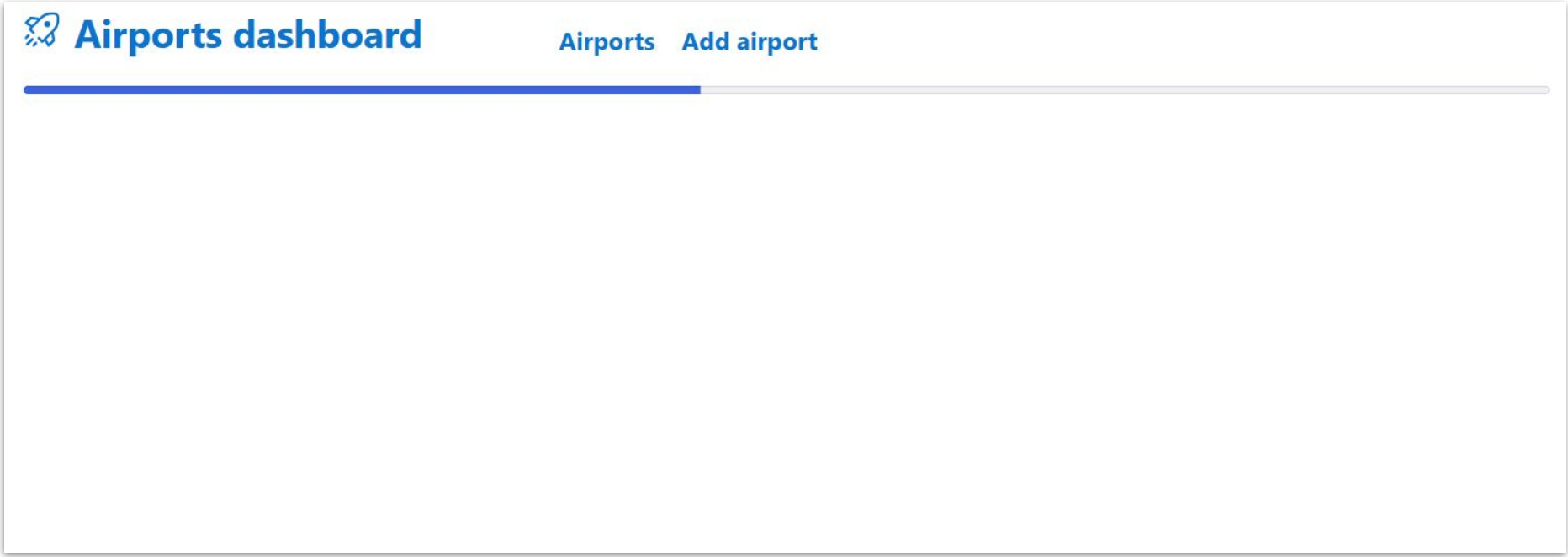
Not mocking of api calls

Not mocking of api calls

- No mocking, as it doesn’t support the test case directly

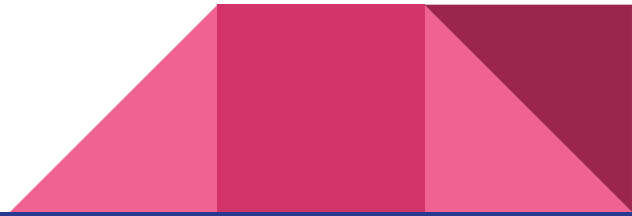


Loader indicator tests



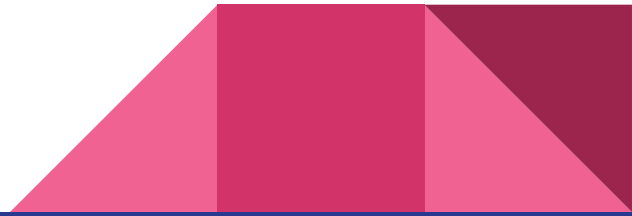
Not mocking of api calls

- No mocking, as it doesn’t support the test case directly
- Async communication means
  the result may appear after test is finished already
- It triggers random like errors on extended test suites



How to fix?

- Don’t use handcrafted patterns (i.e. mocking implementation details)
- Use tooling dedicated to mocking api calls
- Mock all calls triggered during test execution



Mocking with MSW

```ts
// api-handlers/countries.ts
import {http, HttpResponse} from 'msw';
import {CountriesDto} from '../../../src/api/rest/countries.dto';

export const countriesHandler = (responseData: CountriesDto, status = 200) =>
   http.get(`*/api/countries`, () => HttpResponse.json(responseData, {status}));
```

```ts
// AirportView.test.tsx
server.use(
   countriesHandler([countryData]),
   regionsHandler([]),
);
```

How to fix?

- Don’t use handcrafted patterns
- Use tooling dedicated to mocking api calls
- Mock calls triggered during test execution
- Add listener printing log if no mock is found



Listener attached to MSW

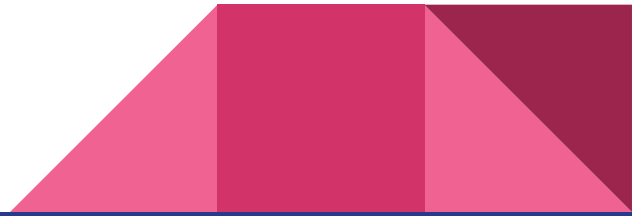
```
beforeAll(() =>
    server.listen({
        onUnhandledRequest ({method, url}) {
            console.error('Found an unhandled %s request to %s' , method,
url);
        },
    }),
);

// Found an unhandled GET request to http://localhost:4000/api/airports/1
```

Overmocking

Overmocking

- Yet another misinterpretation of DRY pattern
- Common mistakes
  - creating mocks on beforeEach() tests group
  - during setup of mocking environment
  - using wildcard mocks, i.e. /airports/*



Hardcoding mocks on MSW setup

```
export const server = setupServer(
   http.get(`*/api/countries`, () => HttpResponse.json(countriesList)),
   http.get(`*/api/airports`, () => HttpResponse.json(airportsList)),
   http.get(`*/api/airports/*`, () => HttpResponse.json(airport))
   // ...
);
```

How to fix it?

- Mock each api call per test



Restrictive mocking

Restrictive mocking

- To mock only api calls triggered during test, per test case
- To not mock all remaining calls - console error on unhandled api call
- It’s important to assert:
  - path params
  - request body shape down to each part
  - other applicable elements, i.e. search params, headers, cookies
- Best pattern for backend integration assertions



Factory pattern

Repetitive mocks creation per test case

```
// Given airport id
const airportId = 1;

// And Airport data returned by api call
const airportData: AirportDto = {
   id: airportId,
   // ...
};

// And GET /airport/{airportId} handler
server.use(airportHandler(airportId, airportData));
```

An idea: Base object for mock shaping

```
// data-mocks.ts
const baseAirport: AirportModel = {
    name: 'test airport',
    iata: 'TES',
    country id: 1,
    regions: []
}

// Component.test.tsx
const spanishAirport = {
    ...baseAirport,
    country_id: 2
}
```

Case of complex data objects

```
const complicatedDto = {
    client: {
        name: 'test name',
        surname: 'test surname'
    },
    contact: {/* ... */},
    address: {
        permanent: {/* ... */},
        temporary: {/* ... */}
    }
}
```

Factory pattern

- Base object for mock shaping
- Spread operator vs. dedicated library



Factory.ts use example

```ts
// factories/countries.ts
import * as Factory from 'factory.ts';

export const countryFactory = Factory.Sync.makeFactory<CountryDto>({
   id: 1,
   name: 'test country name',
   is_in_schengen: true,
});
```

```ts
// AirportView.test.tsx
// And country mock
const country = countryFactory.build({name: 'different country name' });

// And mocks of api calls triggered during the test
server.use(countriesHandler([country]) /* ... */);
```

Factory pattern

- Base object for mock shaping
- Spread operator vs. dedicated library
- Mind a trap: Never trust a base object, always overload items used on tests



Flaky relation to the base object

```
// factories/countries.ts
import * as Factory from 'factory.ts';

export const countryFactory = Factory.Sync.makeFactory<CountryDto>({
   id: 1,
   name: 'test country name',
   is_in_schengen: true,
});
```

```
// AirportView.test.tsx
// And schengen country
const schengenLikeCountry = countryFactory.build({name: 'other country name'});

// And schengen-related test checks
// ...
```

Overloading all items used on a test case

```
export const countryFactory = Factory.Sync.makeFactory<CountryDto>({
    id: 1,
    name: 'test country name',
    is_in_schengen: true,
});
```

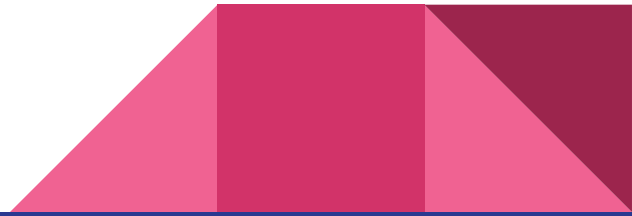
```
// And schengen country
const schengenCountry = countryFactory.build({
    name: 'test country name',
    is_in_schengen: true
});

// And schengen-related test checks
```

Randomization pattern used on mocks creation

Randomization pattern used on mocks creation

- An assumption: entire base object should get randomized
- Effect: Necessity of overloading all data used on test case
- Result: Base object aligned with Dto/Model.
  Test case’s responsibility for data content.
- Necessity to randomize on an enterprise level



Use case of Faker library

```
import * as Factory from 'factory.ts';
import {faker} from '@faker-js/faker';

export const airportFactory = Factory.Sync.makeFactory<AirportDto>({
    id: faker.number.int(),
    name: faker.airline.airport().name,
    iata: faker.airline.airport().iataCode,
    country_id: faker.number.int(),
    regions: faker.helpers.multiple(
        () => faker.number.int(), {count: faker.number.int(5)}
    ),
});
```

Test case data overloading

```
// And country data
const country = {
   id: 1, is_in_schengen: false,
   is_passport_required: true, is_visa_required: false
}
const nonSchengenCountryData  = countryFactory.build(country)

// And airport data
const airport = {id: 99, name: 'test airport name', country: country.id}
const airportData = airportFactory.build(airport)

// And mocks of api calls triggered during the test
await countriesMock(page, [nonSchengenCountryData])
await airportMock(page, airport.id, airportData)

// And test for non-schengen airport ...
```

Pros and cons of using such a pattern?

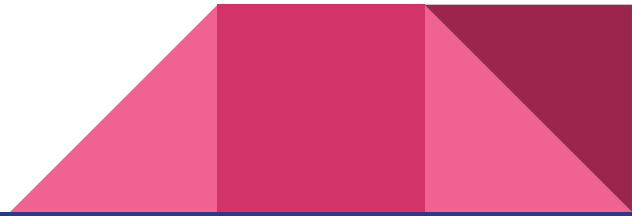
- Cons
  - Mock creation process gets quite complicated
  - So is the test debugging
  - Requires some experience over developing valuable tests
- Pros
  - It mimics the unpredictable nature of users actions best
  - Can randomly find some bug on an app



Playwright tests (functional & e2e)

Functional tests in a nutshell

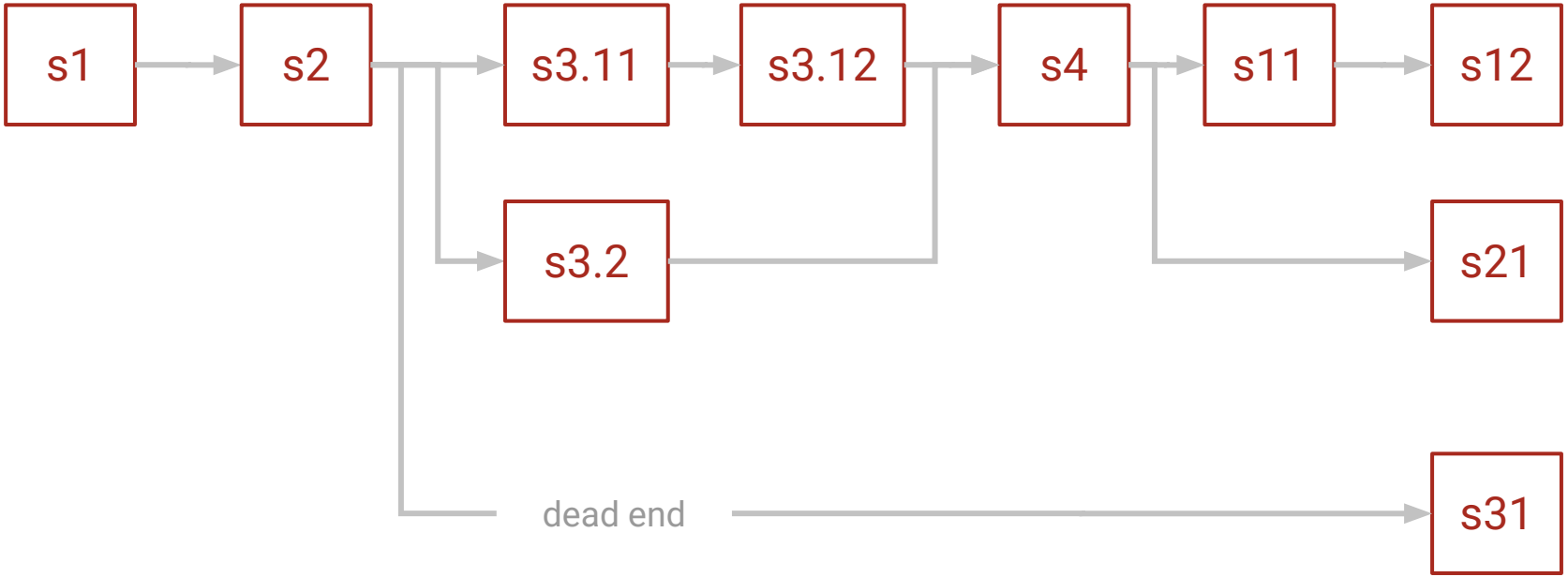
- Should I choose Playwright or Cypress?
- Are functional tests similar to integrations?
  - Yes for single view functionalities
  - No for stepper journeys
- Brilliant gems of Playwright
  - Snapshots (i.e. visual)
  - Integration with services
    - Lighthouse
    - Axe (for a11y)
  - Keyboard navigation



Setup details

Functional test example

Complicated stepper journeys



Accessibility testing

Current state of accessibility in web development

- Accessibility topics are common on frontend conferences / meetups
- June 28th, 2025 - European Accessibility Act requires service compliance at WCAG 2.1 AA level (1)
- 88% of websites are not fully compliant with latest standards (2)(3)
- Because we all are having excuses

(1) https://www.wcag.com/compliance/european-accessibility-act/

(2) https://www.accessibilitychecker.org/research-papers/the-state-of-web-accessibility-in-2024-research-report/

(3) https://webaim.org/projects/million/

Our approach was similar.
But something has changed.



Auditing Design Systems for Accessibility by Anna E. Cook

https://www.youtube.com/watch?v=L0PRzlyP1Zc

“Accessibility is not about being perfect. It’s about trying.”

Anna E. Cook, Wey Wey Web 2023

“It’s best to try a11y testing on new applications. You start small and grow as the project grows.”

Source unknown

What is this a11y really?

A11y is required for users experiencing any of*

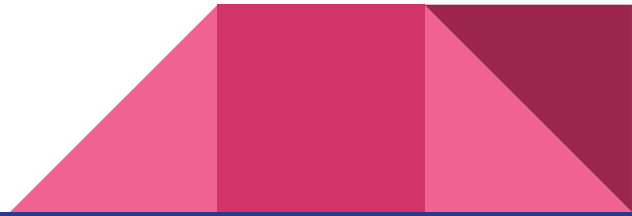
- Visual Impairment
- Hearing Impairment
- Cognitive and Learning Disabilities
- Motor Impairment
- Speech Impairment
- Neurological Limitations



Source: https://reciteme.com/news/common-website-accessibility-issues/

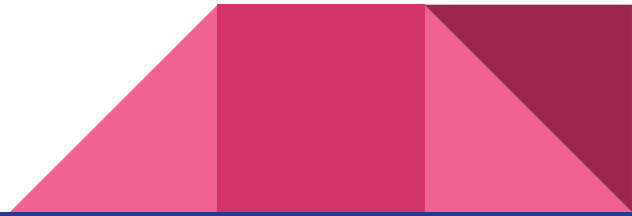
Most common web a11y issues

- Colour contrasts - to cover by design
- Complicated content, structure or user journey - to cover by UX
- Poor element identification - this one is ours
- Problematic interaction - this one too



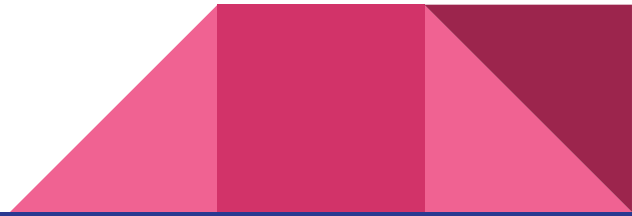
Poor element identification

- Vast possibilities of page presentation
  - Visual presentation (screen)
  - Audio description (screen reader)
  - Via dedicated assistive technologies (i.e. braille reader)
- No/poor description for items like images, links or buttons
- It’s all about quality of HTML produced



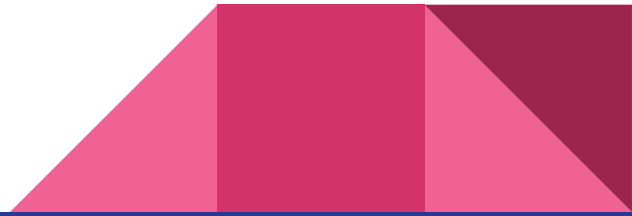
Problematic interaction

- Interacting with the element on the page is:
  - Selecting the element
    - By searching on the page (i.e. getByRole(‘button’, {name: ‘Send’})
    - By using elements list (navigation via Tab key)
  - Performing dedicated action (type, click)
- It’s all about quality of HTML produced



How to approach automatic testing by devs?

- Semantic and well described HTML is the key
- No need to test using all assistive technologies
- It’s all about HTML - we should test production bundle
- Playwright to be the tool of our choice
  (however, it’s not obligatory)



Airports dashboard

Airports Add airport

Add airport

Name

Test airport name

IATA Code

TES

Country

Spain

Regions

island

- [ ] Aegean Islands
- [ ] Corfu Island (Kerkyra)
- [ ] Ha Long Island
- [ ] Rhodes Island
- [x] Balearic Islands
- [ ] Crete Island
- [ ] Ionian Islands
- [ ] Tenerife Island
- [ ] Canary Islands
- [ ] Dodecanese Islands
- [x] Maiorca Island

Vaccination Notes

Submit

Example repository

- Fully functional basis of React SPA + Backend
- Reliable test examples
- Framework and reusables to run tests
- Note: The app is not a11y compliant!
  (But you will find it out on test report)

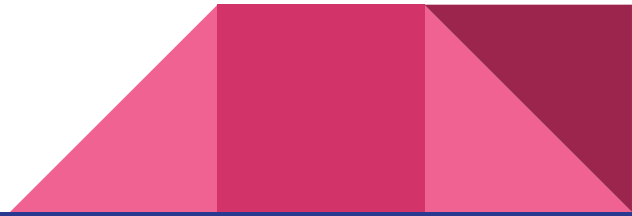


https://github.com/LukaszNowakPL/web-a11y-testing

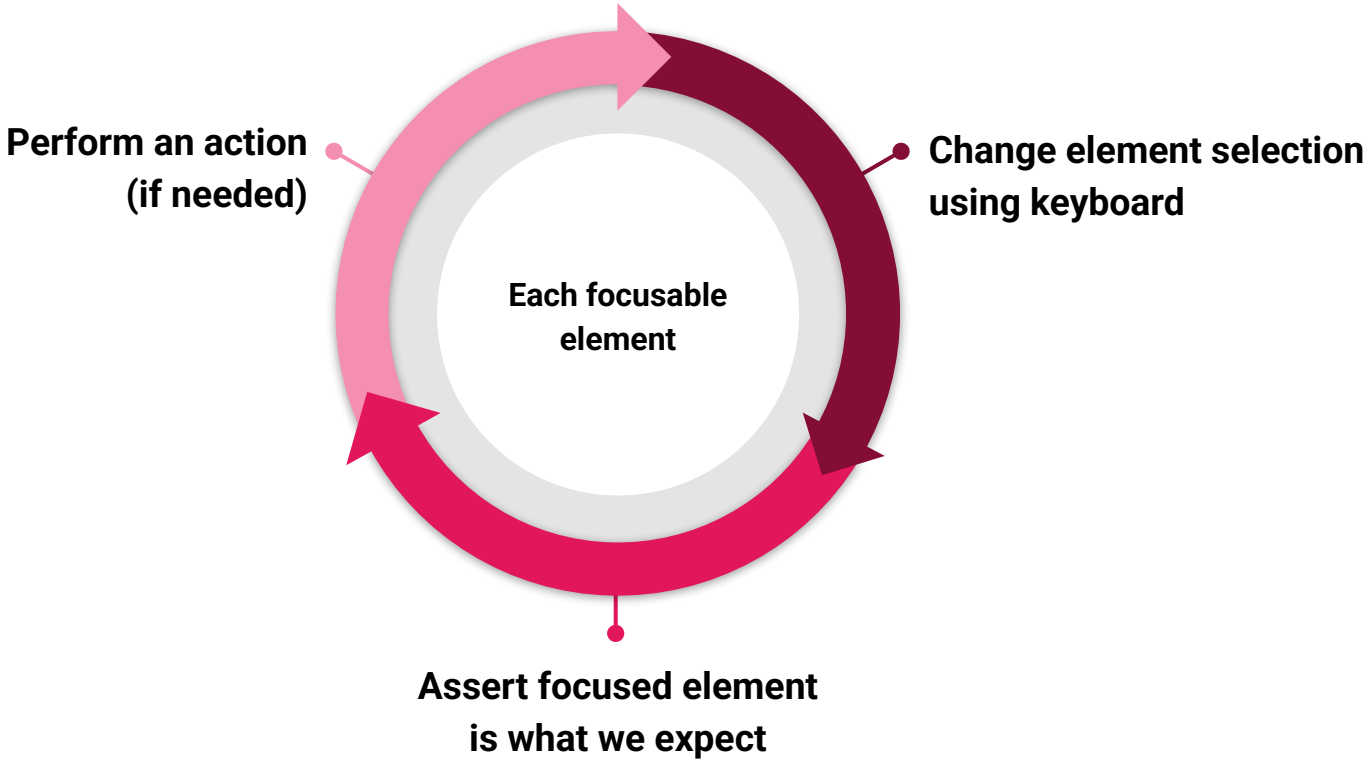
How can we test a11y automatically?

User journey with keyboard navigation

- It’s just another functional test - a low hanging fruit
- Minimal approach: cover at least a simplest happy path user journey



Scenario principle

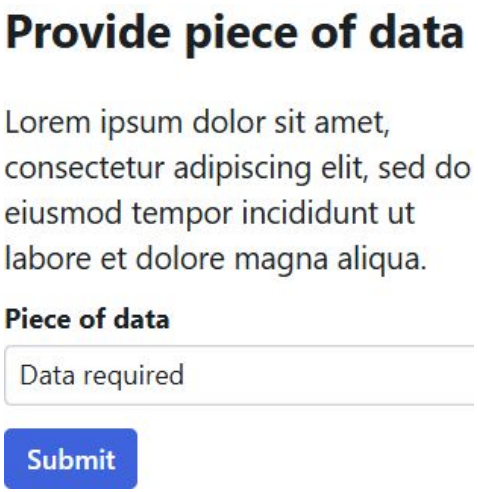


Static HTML audits

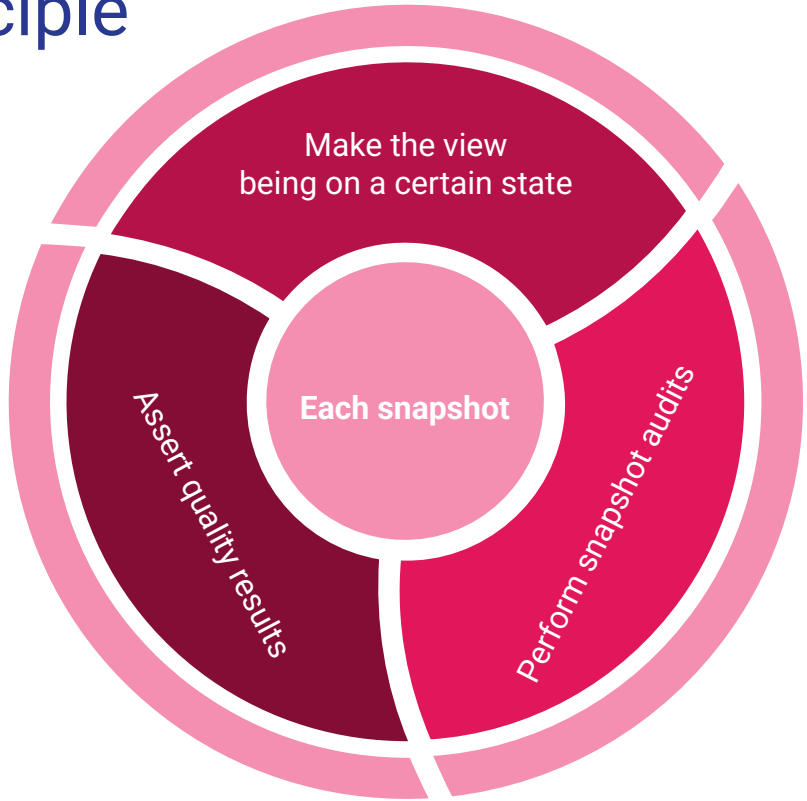
Same page, different states







Approach principle



The audit process

```
public async assertSnapshots(snapshotTitle: string, config?: AssertSnapshotConfig) {
    await Promise.all([
        this.assertAxeChecks(snapshotTitle, Boolean(config?.skipAxeViolationChecks)),
        this.performVisualSnapshotComparison(snapshotTitle),
        this.performAriaSnapshotComparison(snapshotTitle),
        config?.takeLighthouseSnapshot !== undefined
          && this.assertLighthouseSnapshot (snapshotTitle,
config.takeLighthouseSnapshot ),
    ]);
}
```

Lighthouse audit

- playwright-lighthouse for integrated automation
- Accessibility audit is nicely crafted:
  - Best practices, WCAG checks, HTML semantics
  - Visual analysis (colour contrasts, distances, etc.)
- Provides helpful, human readable report



https://www.npmjs.com/package/playwright-lighthouse

Axe accessibility testing engine

- Static code analysis provided by industry leader
- Produces raw, textual, in-depth reports
- Restricts scanning against certain criterias (i.e. up to WCAG 2.1 AA)



https://playwright.dev/docs/accessibility-testing

Aria snapshot

- Accessibility tree idea*
- It is like a DOM dedicated for a11y
- Playwright Aria snapshots functionality
- Accessibility regression pattern



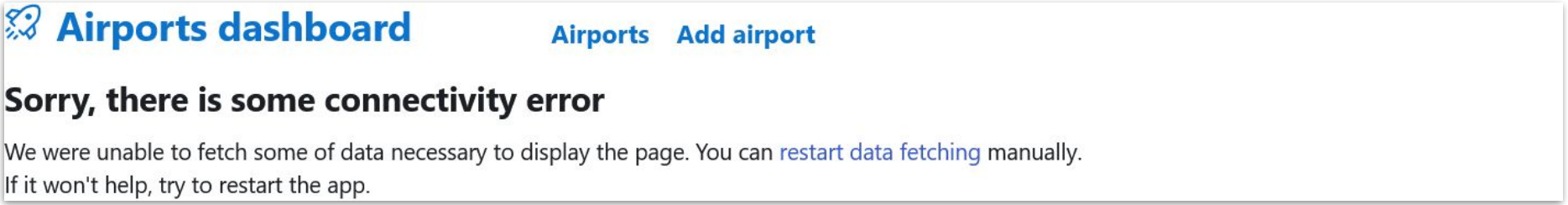
https://playwright.dev/docs/aria-snapshots#aria-snapshots

Source: https://developer.mozilla.org/en-US/docs/Glossary/Accessibility_tree

Aria snapshot example

```
- banner:
 - link "Airports dashboard" :
   - img
   - text: Airports dashboard
 - navigation:
   - list:
     - listitem:
       - link "Airports"
     - listitem:
       - link "Add airport"
- main:
 - alert: Sorry, there is some connectivity error
 - paragraph:
   - text: We were unable to fetch some of data necessary to display the page. You can
   - button "restart data fetching"
   - text: manually.
 - paragraph: If it won't help, try to restart the app.
```

Aria snapshot example



Unknown super-power of aria snapshot

```
    - listitem:
      - link "Airports"
    - listitem:
      - link "Add airport"
- main:
 - heading "Add airport" [level=1]
 - text: Name
 - textbox "Name"
 - text: IATA Code
 - textbox "IATA Code"
 - text: Country
 - combobox "Country": Select one
 - text: Regions
```

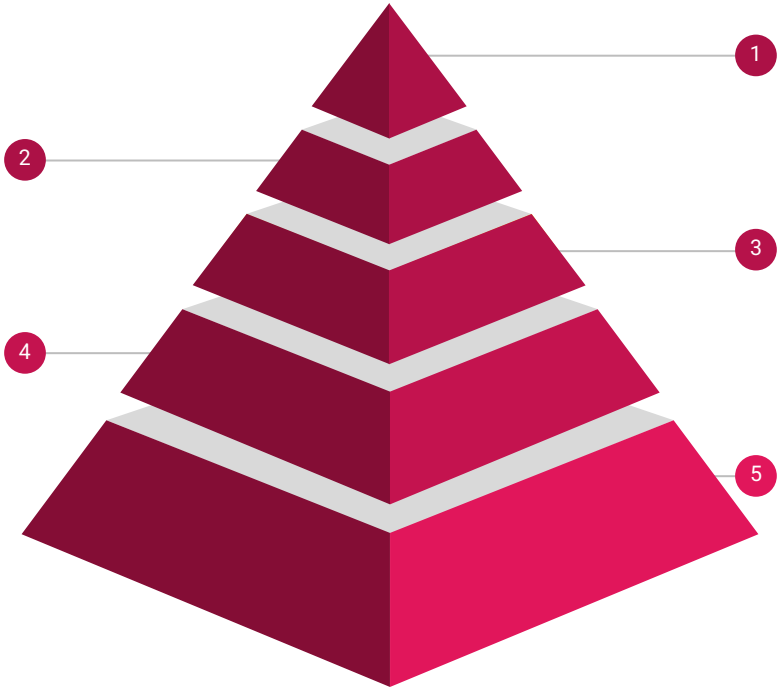
Gotchas and findings

Gotchas and findings

- Snapshot suite is time consuming



Snapshot execution times



Bare functional journey of 60 snapshots

1 8 sec

Aria snapshot regression

9 sec 2

Axe audit

3 44 sec

Lighthouse audit

114 sec 4

Entire snapshot journey

5 132 sec

Gotchas and findings

- Snapshot suite is time consuming
- Snapshot idea helps to understand the view lifecycle
- It’s crucial to cover a11y on design system





Auditing Design Systems for Accessibility by Anna E. Cook

Gotchas and findings

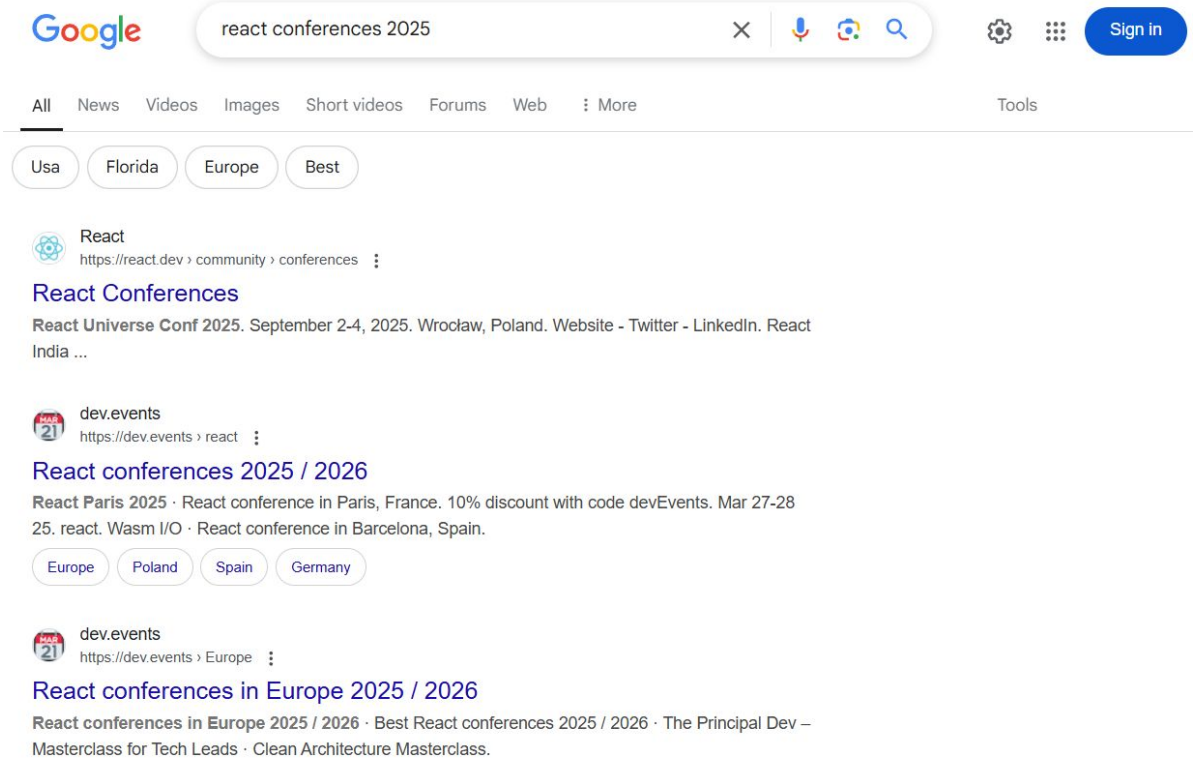
- Snapshot suite is time consuming
- Snapshot idea helps to understand the view lifecycle
- It’s crucial to cover a11y on design system
- Devs to test their own application, while QAs to focus on entire product



App vs. Product scope

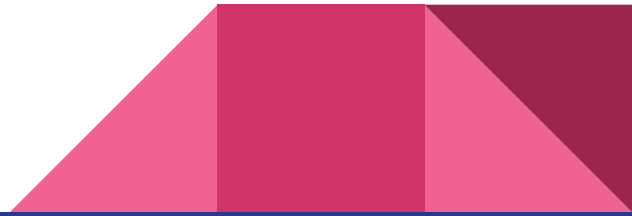
Product section

App section



Gotchas and findings

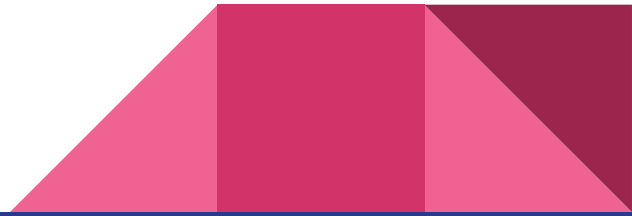
- Snapshot suite is time consuming
- Snapshot idea helps to understand the view lifecycle
- It’s crucial to cover a11y on design system
- Devs to test their own application, while QAs to focus on entire product
- A11y testing is relatively new thing. It lacks global solutions, toolings or good practice definitions
- Beware of paid solutions



Conclusions

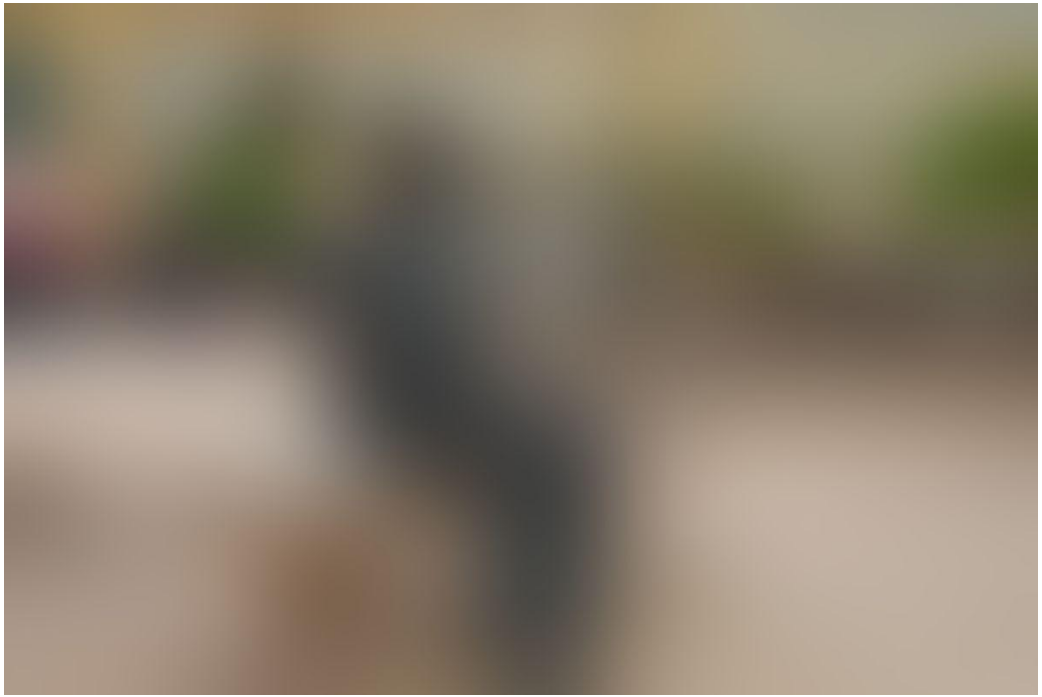
Conclusions

- A11y is not about being perfect. It’s about trying.
- So, we did not become an a11y experts. We are, however, an a11y-aware.
- We are able to detect only small factor of a11y errors
- Tests with real users and assistive technologies is the key



What automation won’t find?

```
<img
  src="picture.png"
  alt="Picture"
/>
```

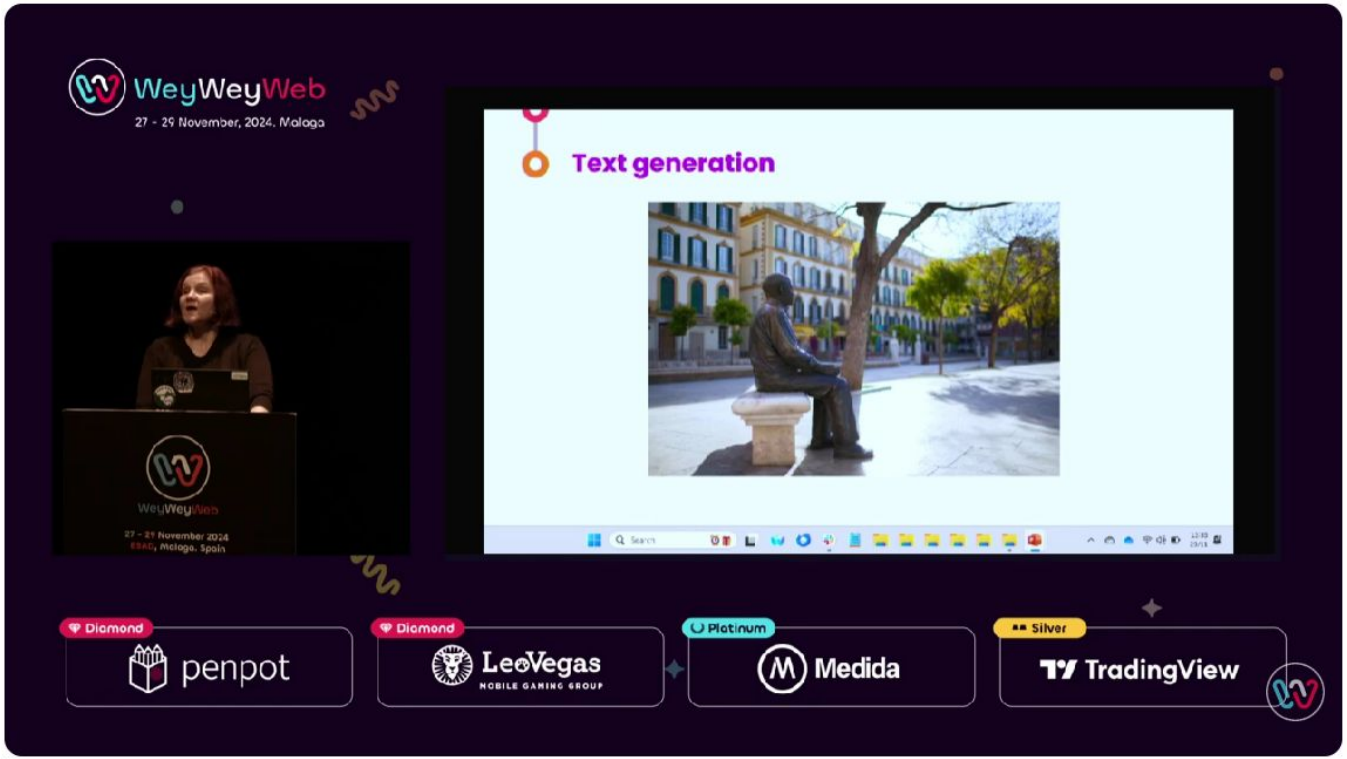


What human will find?

```
<img
  src="picture.png"
  alt="Picture of a Picasso
statue located at Plaza de la
Merced in Malaga"
/>
```



Source: Świat Hegemona (Travel blog)



https://www.youtube.com/watch?v=X8wcbJR9IaU

AI and Accessibility: the Good, the Bad, and the Bollocks by Léonie Watson

Playwright disclaimer

DISCLAIMER

Automated accessibility tests can detect some common accessibility problems such as missing or invalid properties. But many accessibility problems can only be discovered through manual testing. We recommend using a combination of automated testing, manual accessibility assessments, and inclusive user testing.

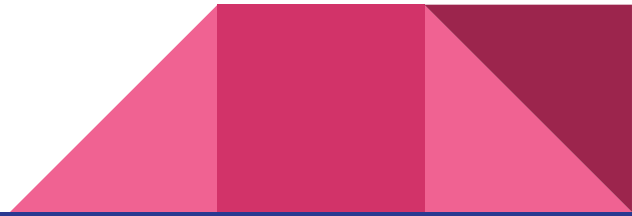
For manual assessments, we recommend Accessibility Insights for Web, a free and open source dev tool that walks you through assessing a website for WCAG 2.1 AA coverage.

Source: https://playwright.dev/docs/accessibility-testing

Accessibility test code examples

Accessibility testing run through

- Keyboard navigation
- Snapshot testing
  - Axe analysis
  - Lighthouse analysis
  - Accessibility tree generation



Visual snapshot comparison

Visual snapshot comparison idea

- Idea of a golden snapshot
- Take a snapshot of a page
- Compare with golden snapshot
  - No differences means everything is ok
  - Differences means we must investigate if it’s expected
- Update golden snapshot if necessary
  - Diff to be verified on MR
  - Has to be approved by business



Visual snapshot comparison challenges

- Different environments create different snapshots
- maxDiffPixelRation escape hatch
- Dockerized solution



Visual snapshot test code examples

Let’s summarize the workshop

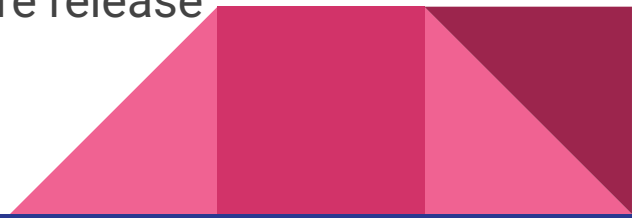
What we went through

- Integration test the bullet proof way
- Enterprise class api mocking
- Functional / e2e tests in Playwright
- Accessibility testing
- Visual regression



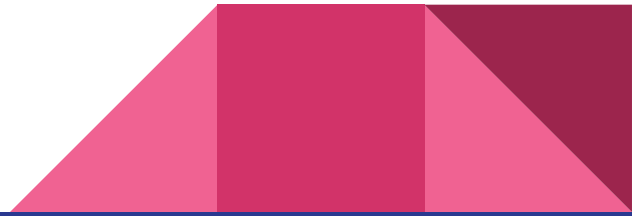
Final recommendation

- Tests should give an assurance of something works as expected
  - Frontend testing is not like any others
  - Simulate the way users use an application
- Integration tests are 80% of works done with 20% effort, only if:
  - they are scoped correctly (rather test of entire views)
  - are independent from implementation details (possibility to upgrade dependencies)
  - mock only an outside world
- It’s best to start from proper integration testing
  - With focus on solid api calls mocking
- Add several functional scenarios and snapshot before release
- A11y testing is a helper, not a solution



Why should you do that

- Latest trend of shift left testing
- Tests scenarios handled by humans, not AI
- Writing tests made me a better frontend dev



Thank you for your attention♡

Q&A

Don’t miss the repository full of testing examples!



https://github.com/LukaszNowakPL/bulletproof-frontend-testing